MsmqCache (mcached) - Production Guide

By Hadi Cahyadi - version 1.0 - 2026-10-09

1. What it is

`mcached` is a **Redis-compatible cache server** written in C# / .NET 8. It listens on a **TCP port (default 6379)** and speaks **RESP2**, so `redis-cli` and normal Redis client libraries can connect. An optional **MSMQ transport** (Windows only) is also available. The bundled `mcache` CLI talks TCP by default.

It is a **strings-only** cache: no lists, hashes, sets, pub/sub, transactions, Lua, replication, clustering, ACL users or RESP3.

> **Status:** the code has been syntax-checked and the tests were written, but nothing has been compiled or run yet (the build environment had no .NET SDK and MSMQ is Windows-only). First step before any real use: `dotnet build`, `dotnet test`, then `python scripts/smoke_test.py` against a running server.

2. Architecture

- **Store**: one dictionary + exact LRU list + TTL index (sorted set), guarded by a single lock. Memory is approximated (2 bytes per char plus overhead per entry). Limits: `MaxMemoryMb`, `MaxKeys`. Policies: `allkeys-lru` or `noeviction` (returns an `OOM` error).
- **TCP transport**: async listener, one task per connection, strict RESP parser (multibulk + inline), pipelining with batched flush, `MaxClients`, optional idle timeout, optional TLS 1.2/1.3, `AUTH` with constant-time compare.
- **MSMQ transport** (optional): request queue plus a per-client private reply queue, JSON bodies, one `MessageQueue` instance per worker.
- **Host**: .NET Generic Host. Runs as a console app, a Windows Service, or under systemd. If a transport fails at startup (for example the port is in use) the host stops with a non-zero exit code.
- **Persistence**: periodic and shutdown snapshots (JSON), written to a temp file, fsync'd, then moved over the old file. A hard crash loses at most `SnapshotIntervalSeconds` of writes.

3. Redis compatibility

Area	Supported
Protocol	RESP2 requests (multibulk and inline), RESP2 replies, pipelining
Data types	Strings only
Commands	PING ECHO GET SET (EX PX NX XX) SETNX SETEX PSETEX MGET MSET GETDEL APPEND STRLEN DEL UNLINK EXISTS INCR DECR INCRBY DECRBY EXPIRE PEXPIRE TTL PTTL PERSIST TYPE KEYS DBSIZE FLUSHDB FLUSHALL SAVE INFO CONFIG AUTH SELECT (0 only) CLIENT (SETNAME GETNAME ID SETINFO) COMMAND QUIT
Values	UTF-8 text. Invalid UTF-8 is rejected with a protocol error, never silently altered
Not supported	HELLO/RESP3, SCAN, lists, hashes, sets, sorted sets, streams, pub/sub, MULTI/EXEC, Lua, replication, cluster

Known limitations: `KEYS` is O(n) and returns at most 10,000 keys. `INFO` does not report `redis_version`, so clients that require it may degrade. Only database 0 exists.

Binary data, pickle and other serializers

Values are **UTF-8 text**, not arbitrary bytes. A `SET` whose value is not valid UTF-8 is rejected with `-ERR Protocol error: value is not valid UTF-8 ...` and the server closes that connection. It never stores a corrupted value.

Value format	Works today?	Note
Plain text, JSON	Yes	Recommended for most uses
Base64 (or hex) of any bytes: pickle, msgpack, protobuf, compressed data, images	Yes	About 33% larger; `MaxValueLength` counts the encoded characters

Value format	Works today?	Note
Raw pickle (protocol 1 and higher), msgpack, protobuf, gzip, images	No	Rejected as described above
Pickle protocol 0 (ASCII only)	Technically yes	Not recommended

Python example storing a pickled object:

```
import base64, pickle, redis

r = redis.Redis(host="127.0.0.1", port=6379)   # add password=... if set

def put(key, obj, ex=None):
    r.set(key, base64.b64encode(pickle.dumps(obj,
protocol=5)).decode("ascii"), ex=ex)

def get(key):
    v = r.get(key)
    return None if v is None else pickle.loads(base64.b64decode(v))
```

Security: `pickle.loads` runs arbitrary code. Only unpickle data from a cache that nobody untrusted can write to: keep the server on loopback, or protect it with a password and TLS. JSON or msgpack is safer for shared caches.

Planned: binary-safe values, so raw bytes round-trip unchanged as in Redis (see the roadmap). Until then, encode to text as shown above.

4. Configuration

The config file is **never** read from the current directory. Lookup order:

1. `--config <path>`
2. environment variable `MSMQCACHE_CONFIG`
3. `%ProgramData%\MsmqCache\config.json` (if it exists)
4. `<exe dir>\config.json` (created with defaults if missing)

`DataDirectory` and `TlsCertificatePath` are relative to the exe folder and expand `%ENVVARS%`.

Main settings

Key	Default	Notes
EnableTcp	true	RESP/TCP transport
BindAddress	127.0.0.1	Use 0.0.0.0 or :: only together with Password and/or TLS
Port	6379	0 = ephemeral (tests)
Password	empty	Redis requirepass equivalent; stored in clear text in the file
MaxClients	10000	Extra connections get an error and are closed
IdleTimeoutSeconds	0	0 = never close idle clients
TlsEnabled / TlsCertificatePath / TlsCertificatePassword	false	.pfx certificate
MaxMemoryMb / MaxKeys	256 / 0	0 = unlimited
EvictionPolicy	allkeys-lru	or noeviction
DefaultTtlSeconds	0	Applied when SET has no TTL
MaxKeyLength / MaxValueLength	512 / 1000000	Characters
PersistenceEnabled / SnapshotIntervalSeconds	true / 60	Snapshot file: data\cache.snapshot.json
DisabledCommands	empty	e.g. FLUSHALL, FLUSHDB, CONFIG, KEYS
EnableMsmq	false	Optional MSMQ transport

Runtime changes (persisted to the file): `CONFIG SET maxmemorymb | maxkeys | evictionpolicy | defaultttlseconds | snapshotintervalseconds | persistenceenabled | password | maxclients | idletimeoutseconds | replyrecoverable | usedeadletterqueue <value>`. Lowering `maxmemorymb` evicts on the next write. Everything else needs a restart. `CONFIG GET` never reveals passwords.

5. Security checklist

- Leave `BindAddress` on loopback unless remote access is really needed.
- For remote access set a long random `Password` **and** enable TLS. Without TLS the password and data travel in clear text.
- Restrict the config file ACL (it contains the password) and the data directory (snapshots contain all values).
- Disable commands you do not need: `DisabledCommands: ["FLUSHALL","FLUSHDB","CONFIG","KEYS"]`.
- Windows: open the firewall only for trusted source addresses (`install-service.ps1 -OpenFirewall` opens the port for all sources, so narrow the rule afterwards).
- Wrong passwords are delayed by 250 ms and the connection is closed after 5 failures. There is no global rate limit, so put the port behind a firewall.
- Run the service under a dedicated low-privilege account (the default is LocalSystem).

6. MSMQ transport (optional)

Use it only when clients are Windows-local or already on MSMQ. TCP is the primary, Redis-like path.

MSMQ feature	Status
Private local queues, timed receive, response queues	Used
Recoverable messages	Option: `ReplyRecoverable` (server), `Recoverable` (client)
Dead-letter queue	Option: `UseDeadLetterQueue` (replies). Counts against the machine quota; purge it
Queue quota and journal	Options: `QueueMaxSizeKb`, `QueueJournal`, `JournalMaxSizeKb`
Queue ACLs	`ApplyAclOnStartup` + `QueueAcl` (send / receive / full). Failure is fatal on purpose

MSMQ feature	Status
Authentication and encryption	`QueueAuthenticate`, `QueueEncryption`, `RequireAuthenticatedRequests`; client `UseAuthentication`, `UseEncryption`. Need a domain (AD) or internal certificates
Transactions (MSDTC), priority, public queues / AD, HTTP and multicast, triggers, clustering	Not used

Limits: 4 MB maximum message size; non-recoverable messages are lost if the MSMQ service restarts; remote machines are untested.

Operational warnings: the December 2025 Windows update (KB5071546) broke MSMQ sends on some Windows 10 and Server 2012-2019 systems (storage folder permissions). Test on your patched hosts. `Experimental.System.Messaging` is a community port, not an official Microsoft client.

7. Operations

Install (Windows): `scripts\package.ps1`, unzip, then `install-service.ps1 -Dir <server folder>`. The service auto-starts and restarts on failure (5 s, 5 s, 30 s).

Install (Linux): extract the tar.gz, copy `scripts/mcached.service` to `/etc/systemd/system/`, adjust paths and user, then `systemctl enable --now mcached`.

Monitoring: `INFO` reports clients, connections, rejected connections, keys, keys with TTL, approximate memory, hits, misses, expired and evicted keys, total commands and last save time. Logs go to the console, the Windows Event Log (service mode) or journald.

Backups: copy `data\cache.snapshot.json` (it is replaced atomically, so a copy is always consistent). A corrupt snapshot is renamed `.corrupt-<timestamp>` and the server starts empty.

Sizing: memory accounting is approximate (about 2 bytes per character plus 96 bytes per entry). Leave headroom below the machine's RAM.

Verification after install:

1. `python scripts/smoke_test.py --port 6379 [--password PW]` (raw RESP, no client library)

2. `redis-cli -p 6379 PING`
3. `mcache INFO`

8. Testing and CI

- `StoreTests`: TTL, LRU, limits, increments, glob, snapshot round-trip.
- `TcpServerTests` / `TcpAuthTests`: real loopback TCP against the real host: RESP framing, inline commands, pipelining, soft vs protocol errors, UTF-8 handling, concurrency, auth, hidden password.
- `MsmqTransportTests`: skipped unless `MSMQCACHE_MSMQ_TESTS=1` on a Windows host with MSMQ running.
- Workflows: **ci** (build + test on Windows and Ubuntu), **build** (package win-x64 and linux-x64, framework-dependent and self-contained), **release** (tag `vX.Y.Z` -> GitHub release with archives and SHA-256 files), **integration-msmq** (manual / weekly, not a release gate).

9. Licensing

- **MsmqCache**: MIT.
- **MSMQ**: a Windows component with no license that your code inherits; use is governed by your Windows license.
- **Experimental.System.Messaging 1.2.0**: Microsoft Reciprocal License (Ms-RL), per its NuGet page and repository. Ms-RL is file-level copyleft: only the library's own files stay Ms-RL, and your files can be MIT. Versions up to 1.1.0 were labelled MIT by mistake, so do not rely on them. Binary distributions must carry the notices: `THIRD-PARTY-NOTICES.md` (with the full Ms-RL text and the source link) is included in every release archive.
- **.NET libraries and runtime**: MIT, notice included.
- This is not legal advice; have counsel review before commercial distribution.

10. Roadmap (not implemented)

- Hashes, lists, sets and `SCAN`
- RESP3 / `HELLO`
- ACL users instead of a single password

- Replication / persistence log (AOF-style)
- Binary-safe values (currently UTF-8 text)
- Metrics endpoint (Prometheus)
- Windows performance counters for the MSMQ side